From Model to Medium

Topic 3: FastAPI (Python Web Framework)

ML + UI + API + MCP Project

Learning Path

October 26, 2026

Outline

1. What is FastAPI?
2. Our first endpoints
3. The Game Power API
4. Docs, Running, Testing
5. Takeaways

Why FastAPI for our ML project?

Key idea

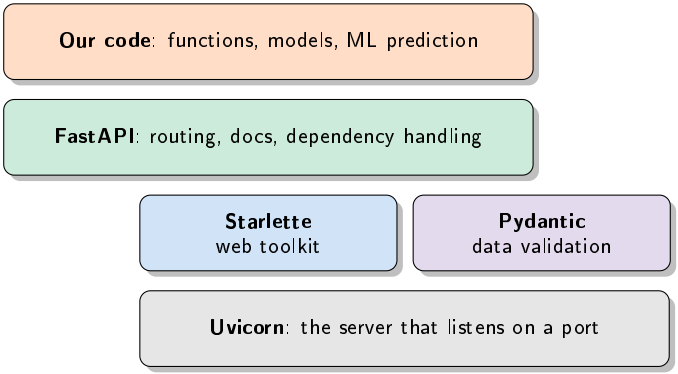
FastAPI turns ordinary Python functions into web endpoints, and builds validation and documentation from the type hints.

ML application

Our trained model becomes one function: `POST /v1/predictions` receives stats and returns the predicted power.

- **Fast to write**: a few lines per endpoint.
- **Automatic validation**: wrong types return 422.
- **Free docs**: Swagger UI at `/docs`.
- **Python-native**: works with scikit-learn, pandas, joblib.

The stack under the hood



Hello World

```python
from fastapi import FastAPI

app = FastAPI(title="Hello API")

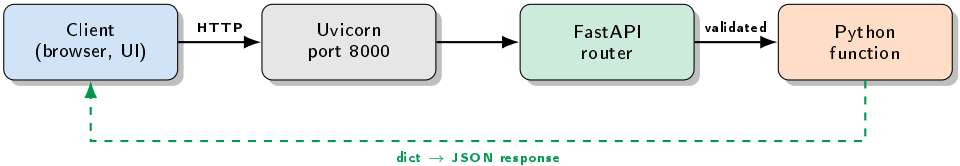

@app.get("/")
def hello():
    return {"message": "Hello, player one!"}
```

1. `FastAPI()` creates the app.
2. The **decorator** links method + URL to the function.
3. A returned `dict` becomes **JSON**.

Response

```
200 OK
{"message": "Hello, player one!"}
```

From request to function



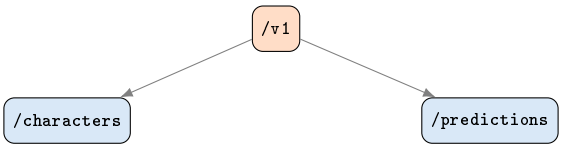
Path and query parameters

```python
@app.get("/characters/{character_id}")                    # path parameter
def one_character(character_id: int, verbose: bool = False):    # query parameter
    data = {"id": character_id, "name": "pikachu"}
    if verbose:
        data["stats"] = {"attack": 55, "defense": 40, "speed": 90}
    return data
```

Request	Result	Why
/characters/25	200, basic data	verbose defaults to False
/characters/25?verbose=true	200, with stats	query parameter read
/characters/abc	422	abc is not an int

Topic 2 contract → code

Route	Function	Success / error
`GET /v1/characters`	`list_characters`	200, paginated
`GET /v1/characters/{id}`	`get_character`	200 / 404
`POST /v1/predictions`	`create_prediction`	200 / 422



Pagination and 404 in FastAPI

```python
@app.get("/v1/characters")
def list_characters(page: int = Query(1, ge=1), limit: int = Query(5, ge=1, le=50)):
    items = list(CHARACTERS.values())
    start = (page - 1) * limit
    return {"items": items[start:start + limit],
            "page": page, "limit": limit, "total": len(items)}


@app.get("/v1/characters/{character_id}")
def get_character(character_id: int):
    if character_id not in CHARACTERS:
        raise HTTPException(
            status_code=404,
            detail={"code": "CHARACTER_NOT_FOUND",
                    "message": f"Character {character_id} does not exist"})
    return CHARACTERS[character_id]
```

The prediction endpoint

```python
from pydantic import BaseModel

class PredictionRequest(BaseModel):
    attack: int
    defense: int
    speed: int

class PredictionResponse(BaseModel):
    predicted_power: float
    model_version: str

@app.post("/v1/predictions",
          response_model=PredictionResponse)
def create_prediction(request: PredictionRequest):
    power = (1.5 * request.attack + request.defense
             + 0.5 * request.speed)
    return {"predicted_power": power,
            "model_version": "0.1.0"}
```

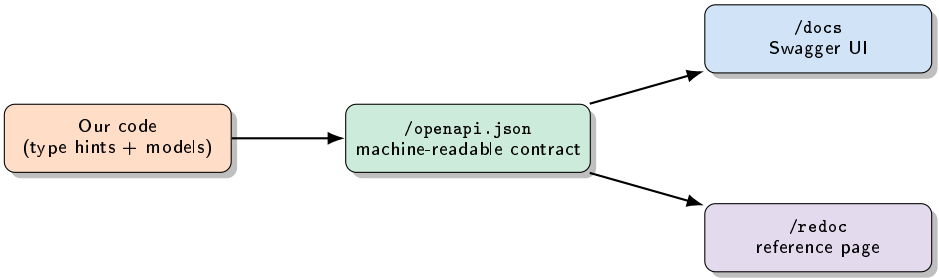
Today vs. Topic 5

The formula is a placeholder. Later the body becomes `model.predict(...)` with our trained regression.

Body type

The function parameter typed as a Pydantic class tells FastAPI to read and validate the **JSON body**.

Automatic documentation



Remember

Change the code and the docs update themselves: documentation can never be out of date.

Running the server

```
# in the folder that contains 03_game_api.py
uvicorn 03_game_api:app --reload --port 8000
```

Part	Meaning	Tip
03_game_api	Python file (no .py)	run from that folder
app	variable inside the file	name must match
-reload	restart on every save	development only
-port 8000	where to listen	change if busy

Then open `http://127.0.0.1:8000/docs` and use **Try it out**.

Testing without a server: TestClient

```python
from fastapi.testclient import TestClient
from game_api import app

client = TestClient(app)

def test_prediction():
    r = client.post("/v1/predictions",
                    json={"attack": 55, "defense": 40, "speed": 90})
    assert r.status_code == 200
    assert r.json()["predicted_power"] == 167.5

def test_missing_character():
    assert client.get("/v1/characters/99").status_code == 404
```

Status codes we get for free

Situation	Code	Who decides
Valid request	200	our function returns
Character not found	404	we raise `HTTPException`
Text instead of a number	422	FastAPI + Pydantic
GET on a POST route	405	FastAPI
Unknown URL	404	FastAPI

Takeaways

1. **Decorator + function** = endpoint; the return value becomes JSON.
2. **Type hints** validate path, query and body data automatically.
3. **Pydantic models** describe the JSON body and the response shape.
4. **Uvicorn** runs the server; `-reload` helps while learning.
5. **/docs** and **TestClient** let us try and test without extra tools.
6. Next topic: **Data Validation with Pydantic**.